

ISET Sidi Bouzid

Département Informatique

Développement Mobile Android avec Java

Cours 03

LES ACTIVITÉS ET LES VUES

Gestion des écrans, cycle de vie, composantes graphiques et dispositions

Enseignant : Kheireddine Derbali

Année universitaire : 2026/2027

Département : Département Informatique

Année universitaire 2026/2027

1. Les Activités (Activity)

Qu'est-ce qu'une Activité ?

Une **Activité** (Activity) est la composante principale d'une application Android. Elle représente l'implémentation métier d'un écran utilisateur, permettant de gérer l'ensemble des vues et des ressources associées.

1.1. Caractéristiques fondamentales

- **Interface utilisateur** : Une activité correspond généralement à un écran complet. Elle peut être avec ou sans interface graphique.
- **Multiplicité** : Un projet Android contient généralement plusieurs activités (ex : `MessagesActivity`, `ComposeActivity`, `SettingsActivity`).
- **Déclaration obligatoire** : Toute activité créée doit obligatoirement être déclarée dans le fichier manifeste `AndroidManifest.xml`.
- **Gestion événementielle** : Une activité réagit à de nombreux événements système représentés par des méthodes de rappel (*callbacks*).

1.2. Structure et Code Java d'une Activité

En Java, une activité hérite de la classe de base `AppCompatActivity` (qui dérive d' `Activity`), offrant un support de compatibilité pour la barre d'action (*ActionBar*).

```
1 package com.isetsbz.tp;
2
3 import android.os.Bundle;
4 import androidx.appcompat.app.AppCompatActivity;
5
6 public class MainActivity extends AppCompatActivity {
7     @Override
8     protected void onCreate(Bundle savedInstanceState) {
9         super.onCreate(savedInstanceState);
10        setContentView(R.layout.activity_main);
11    }
12 }
```

- **Héritage** : La clause `extends AppCompatActivity` permet de réutiliser toute l'infrastructure standard d'un écran Android.
- **Annotation `@Override`** : Indique la redéfinition d'une méthode de la classe mère.
- **Méthode `onCreate()`** : Point d'entrée de l'activité, appelée lors de l'initialisation et du démarrage.
- **`super.onCreate(savedInstanceState)`** : Initialise la partie parente de l'activité.
- **Objet `Bundle`** : Contient l'état sauvegardé de l'application pour restauration. Il vaut `null` lors d'un lancement normal.
- **`setContentView(R.layout.activity_main)`** : Attache le fichier de disposition XML à l'activité Java.

2. Le Cycle de Vie d'une Activité

Une activité Android ne s'exécute pas en continu : elle passe par différents états contrôlés par le système Android (création, affichage, mise en pause, destruction).

2.1. Les 7 Méthodes de Callback

2.2. Paires de callback et phases du cycle

- **`onCreate()` / `onDestroy()`** : Gère la durée de vie globale de l'activité.
- **`onStart()` / `onStop()`** : Gère la phase d'affichage (visibilité).

Méthode	Rôle et Moment d'Exécution
<code>onCreate()</code>	Appelée à la création initiale de l'activité. Initialise les vues et variables.
<code>onStart()</code>	Appelée lorsque l'activité devient visible à l'utilisateur.
<code>onResume()</code>	Appelée lorsque l'activité passe au premier plan et devient interactive .
<code>onPause()</code>	Appelée lorsque l'activité perd le focus (partiellement masquée ou alerte).
<code>onStop()</code>	Appelée lorsque l'activité n'est plus du tout visible (masquée).
<code>onDestroy()</code>	Appelée juste avant que l'activité ne soit complètement détruite de la mémoire.
<code>onRestart()</code>	Appelée lorsque l'activité stoppée est relancée avant repassage par <code>onStart()</code> .

— `onPause()` / `onResume()` : Gère la phase d'interaction (focus utilisateur).

2.3. Scénarios de transition courants

1. **Premier lancement** : `onCreate()` → `onStart()` → `onResume()`.
2. **Appui sur le bouton Home / Appel entrant** : `onPause()` → `onStop()`.
3. **Retour sur l'application** : `onRestart()` → `onStart()` → `onResume()`.
4. **Appui sur le bouton Retour** : `onPause()` → `onStop()` → `onDestroy()`.
5. **Affichage d'une boîte de dialogue (AlertDialog)** : `onPause()`.

Démo : Traçabilité des états dans la console Logcat

Pour observer les transitions d'états, on redéfinit chaque méthode de callback en ajoutant un message de journalisation :

```

1  @Override
2  protected void onStart() {
3      super.onStart();
4      Log.i("Etat Activite", "Activite en etat Start");
5  }
6
7  @Override
8  protected void onResume() {
9      super.onResume();
10     Log.i("Etat Activite", "Activite en etat Resume");
11 }
12
13 @Override
14 protected void onPause() {
15     super.onPause();
16     Log.i("Etat Activite", "Activite en etat Pause");
17 }
18
19 @Override
20 protected void onStop() {
21     super.onStop();
22     Log.i("Etat Activite", "Activite en etat Stop");
23 }
24
25 @Override
26 protected void onDestroy() {
27     super.onDestroy();
28     Log.i("Etat Activite", "Activite en etat Destroy");
29 }

```

3. Les Vues (Views)

3.1. Hiérarchie des Vues

Tous les éléments graphiques de l'interface utilisateur dérivent de la classe abstraite `android.view.View`.

- **Vue simple (View)** : Composant graphique élémentaire (ex : `TextView`, `Button`, `ImageView`).
- **Vue composite (ViewGroup)** : Conteneur de vues capable d'organiser l'emplacement des vues enfants (Layouts).

3.2. Modes de Création des Vues

L'interface utilisateur peut être construite de deux façons :

1. **Statique (Recommandée)** : En créant un fichier de disposition XML dans le dossier `res/layout/`.
2. **Dynamique** : Directement en code Java en instanciant les classes de vues et en les ajoutant à des conteneurs.

3.3. Composants Graphiques Usuels

3.3.1. Affichage de texte : `TextView`

```
1 <!-- En XML (activity_main.xml) -->
2 <TextView
3     android:id="@+id/txtMessage"
4     android:layout_width="wrap_content"
5     android:layout_height="wrap_content"
6     android:text="Bonjour Android"
7     android:textSize="24sp"
8     android:textStyle="bold" />

1 // En Java (MainActivity.java)
2 TextView txtMessage = findViewById(R.id.txtMessage);
3 txtMessage.setText("Bonjour Tout le Monde !");
```

3.3.2. Affichage d'images : `ImageView`

```
1 <!-- En XML -->
2 <ImageView
3     android:id="@+id/imgLogo"
4     android:layout_width="150dp"
5     android:layout_height="150dp"
6     android:src="@drawable/logo" />

1 // En Java
2 ImageView imgLogo = findViewById(R.id.imgLogo);
3 imgLogo.setImageResource(R.drawable.logo_iset);
```

3.3.3. Saisie de texte : `EditText`

```
1 <!-- En XML -->
2 <EditText
3     android:id="@+id/editName"
4     android:layout_width="match_parent"
5     android:layout_height="wrap_content"
6     android:hint="Entrez votre nom"
7     android:inputType="textPersonName" />
```

```

1 // Récupération de la valeur saisie en Java
2 EditText editName = findViewById(R.id.editName);
3 String nom = editName.getText().toString();

```

3.3.4. Boutons de commande et événements : Button

```

1 <!-- En XML -->
2 <Button
3     android:id="@+id/btnSubmit"
4     android:layout_width="wrap_content"
5     android:layout_height="wrap_content"
6     android:text="Valider" />

```

```

1 // Attachement d'un écouteur d'événements (Listener) en Java
2 Button btnSubmit = findViewById(R.id.btnSubmit);
3 btnSubmit.setOnClickListener(new View.OnClickListener() {
4     @Override
5     public void onClick(View v) {
6         Toast.makeText(MainActivity.this, "Clic détecté !", Toast.LENGTH_SHORT).show();
7     }
8 });

```

4. Les Gabarits de Disposition (Layouts)

Les **Layouts** sont des sous-classes de **ViewGroup** chargées d'organiser la disposition spatiale des objets graphiques.

4.1. Gabarit Linéaire : LinearLayout

Le **LinearLayout** aligne ses éléments enfants sur une seule ligne (**horizontal**) ou une seule colonne (**vertical**).

```

1 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
2     android:layout_width="match_parent"
3     android:layout_height="match_parent"
4     android:orientation="vertical"
5     android:gravity="center"
6     android:padding="16dp">
7
8     <!-- Les composants s'empilent verticalement -->
9
10 </LinearLayout>

```

Attribut Principale	Description
android:orientation	Définit l'alignement : vertical ou horizontal .
android:gravity	Aligne le contenu à l'intérieur de la vue (ex : center , right).
android:layout_gravity	Aligne la vue elle-même par rapport à son conteneur parent.
android:layout_weight	Détermine la proportion de l'espace libre occupée par le composant.
android:padding	Spécifie la marge interne (espace entre le bord et le contenu).
android:layout_margin	Spécifie la marge externe (espace entre le bord et les autres composants).

4.2. Gabarit Relatif : RelativeLayout

Le **RelativeLayout** permet de positionner les composantes graphiques les unes par rapport aux autres ou par rapport au composant parent.

Attribut RelativeLayout	Rôle / Positionnement
android:layout_alignParentTop	Aligne le haut du composant sur le haut du parent (true/false).
android:layout_centerInParent	Centre le composant horizontalement et verticalement dans le parent.
android:layout_toRightOf	Place le composant à la droite du composant spécifié par son ID.
android:layout_below	Place le composant en dessous du composant spécifié par son ID.
android:layout_alignTop	Aligne le haut avec le haut du composant référencé.

4.3. Gabarit Tableau : `TableLayout`

`TableLayout` agence ses composantes en lignes et colonnes, de manière semblable à une table HTML. Les lignes sont définies par des balises `<TableRow>`.

- `android:layout_span` : Fusionne plusieurs colonnes pour une vue (équivalent de `colspan`).
- `android:layout_column` : Place explicitement une vue dans un indice de colonne spécifié (début à 0).
- `android:stretchColumns` : Indique les colonnes à étirer pour occuper l'espace restant (ex : "*").
- `android:collapseColumns` : Indique les colonnes à masquer.

4.4. Zone Défilante : `ScrollView`

Lorsque le contenu dépasse la taille physique de l'écran, on enveloppe la disposition dans une balise `<ScrollView>`.

Règle importante pour `ScrollView`

Un `ScrollView` ne peut contenir qu'un **seul enfant direct**. Il faut donc placer un conteneur unique (comme un `LinearLayout`) directement sous le `ScrollView`.

4.5. Inclusion et Réutilisation d'Interfaces : `include` et `merge`

Pour réutiliser des fragments d'interface utilisateur :

- `<include layout="@layout/entete" />` : Permet d'insérer un fichier de disposition XML dans un autre.
- `<merge>` : S'utilise comme élément racine dans le layout inclus pour supprimer les niveaux d'imbrication inutiles de conteneurs.

5. Exercice Pratique : Interface d'Interaction

Énoncé de l'exercice

On souhaite réaliser un écran comportant : 1. Un titre d'accueil "Bonjour". 2. Un texte d'instruction "Entrez votre prénom :". 3. Un champ de saisie de texte. 4. Un bouton "Click". 5. Au clic sur le bouton, afficher un message Toast disant : "Bonjour [Prénom]!".

5.1. 1. Fichier de Disposition XML (res/layout/activity_main.xml)

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_width="match_parent"
4     android:layout_height="match_parent"
5     android:orientation="vertical"
6     android:gravity="center"
7     android:padding="24dp">
8
9     <TextView
10         android:id="@+id/txtTitle"
11         android:layout_width="wrap_content"
12         android:layout_height="wrap_content"
13         android:text="Bonjour"
14         android:textSize="28sp"
15         android:textStyle="bold" />
16
17     <TextView
18         android:id="@+id/txtInstruction"
19         android:layout_width="wrap_content"
20         android:layout_height="wrap_content"
21         android:layout_marginTop="16dp"
22         android:text="Entrez votre prénom : " />
23
24     <EditText
25         android:id="@+id/editName"
26         android:layout_width="match_parent"
27         android:layout_height="wrap_content"
28         android:layout_marginTop="8dp"
29         android:inputType="textPersonName" />
30
31     <Button
32         android:id="@+id/btnHello"
33         android:layout_width="wrap_content"
34         android:layout_height="wrap_content"
35         android:layout_marginTop="16dp"
36         android:text="Click" />
37
38 </LinearLayout>

```

5.2. 2. Fichier Code Java (MainActivity.java)

```

1 package com.isetsbz.tp;
2
3 import android.os.Bundle;
4 import android.view.View;
5 import android.widget.Button;
6 import android.widget.EditText;
7 import android.widget.Toast;
8 import androidx.appcompat.app.AppCompatActivity;

```

```
9
10 public class MainActivity extends AppCompatActivity {
11
12     @Override
13     protected void onCreate(Bundle savedInstanceState) {
14         super.onCreate(savedInstanceState);
15         setContentView(R.layout.activity_main);
16
17         // 1. Recuperation des vues
18         EditText editName = findViewById(R.id.editName);
19         Button btnHello = findViewById(R.id.btnHello);
20
21         // 2. Gestion du clic sur le bouton
22         btnHello.setOnClickListener(new View.OnClickListener() {
23             @Override
24             public void onClick(View v) {
25                 String name = editName.getText().toString();
26                 Toast.makeText(
27                     MainActivity.this,
28                     "Bonjour " + name + " !",
29                     Toast.LENGTH_LONG
30                 ).show();
31             }
32         });
33     }
34 }
```

6. Quiz de Validation

1. Une Activity représente principalement :
 - A. Une image
 - B. Un écran avec un rôle d'interaction et de traitement
 - C. Une base de données
 - D. Un fichier de configuration XML
2. Dans quel fichier une Activity doit-elle obligatoirement être déclarée ?
 - A. activity_main.xml
 - B. strings.xml
 - C. AndroidManifest.xml
 - D. MainActivity.java
3. Quelle méthode permet d'associer une Activity à son interface XML ?
 - A. findViewById()
 - B. setContentView()
 - C. setText()
 - D. onClick()
4. Quel est l'ordre chronologique exact lors du démarrage d'une Activité ?
 - A. onResume → onCreate → onStart
 - B. onCreate → onStart → onResume
 - C. onStart → onDestroy → onResume
 - D. onCreate → onResume → onStart
5. Quelle méthode permet de récupérer une référence de vue XML dans le code Java ?

- A. `getView()`
- B. `findViewById()`
- C. `getXML()`
- D. `searchView()`